Retail Sales & Customer Analytics Dashboard

Project 2 – Business Data Analytics Portfolio Project

This notebook creates a realistic retail sales dataset, cleans and analyzes the data, calculates business KPIs, performs customer segmentation, generates charts, and exports files for Power BI and SQLite.

Tools Used

- Python
- Pandas
- NumPy
- Matplotlib
- SQLite
- Power BI

Main Business Questions

1. How much revenue and profit did the business generate?
2. Which product categories and regions perform best?
3. How do sales change over time?
4. Who are the most valuable customers?
5. Which customer segments should the business target?

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sqlite3
import os
import warnings

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

Create a Realistic Retail Sales Dataset

```
np.random.seed(42)

number_of_orders = 5000

dates = pd.date_range(
    start="2023-01-01",
    end="2025-12-31",
    freq="D"
)

regions = ["Ontario", "Quebec", "British Columbia", "Alberta", "Manitoba"]
categories = ["Electronics", "Furniture", "Office Supplies", "Clothing"]
customer_segments = ["Consumer", "Corporate", "Small Business"]

products = {
    "Electronics": ["Laptop", "Monitor", "Keyboard", "Mouse", "Headphones"],
    "Furniture": ["Office Chair", "Desk", "Bookshelf", "Cabinet", "Lamp"],
    "Office Supplies": ["Notebook", "Printer Paper", "Pens", "Stapler", "Folders"],
    "Clothing": ["Jacket", "T-Shirt", "Jeans", "Shoes", "Sweater"]
}

price_ranges = {
    "Electronics": (35, 1800),
    "Furniture": (50, 900),
    "Office Supplies": (5, 120),
    "Clothing": (20, 250)
}

```

```
rows = []

for order_number in range(1, number_of_orders + 1):
    category = np.random.choice(categories, p=[0.32, 0.23, 0.25, 0.20])
    product = np.random.choice(products[category])
    minimum_price, maximum_price = price_ranges[category]

    unit_price = round(np.random.uniform(minimum_price, maximum_price), 2)
    quantity = np.random.randint(1, 6)
    discount = np.random.choice([0, 0.05, 0.10, 0.15, 0.20], p=[0.35, 0.20, 0.20, 0.15, 0.10])

    revenue = unit_price * quantity * (1 - discount)
    cost_ratio = np.random.uniform(0.55, 0.82)
    cost = revenue * cost_ratio
    profit = revenue - cost

    customer_id = f"CUST-{np.random.randint(1, 1001):04d}"

    rows.append({
        "Order_ID": f"ORD-{order_number:05d}",
        "Order_Date": np.random.choice(dates),
        "Customer_ID": customer_id,
        "Customer_Segment": np.random.choice(customer_segments, p=[0.60, 0.25, 0.15]),
        "Region": np.random.choice(regions),
        "Category": category,
        "Product": product,
        "Quantity": quantity,
        "Unit_Price": unit_price,
        "Discount": discount,
        "Revenue": round(revenue, 2),
        "Cost": round(cost, 2),
        "Profit": round(profit, 2)
    })

sales_df = pd.DataFrame(rows)

sales_df.head()
```

	Order_ID	Order_Date	Customer_ID	Customer_Segment	Region	Category	Product	Quantity	Unit_Price	Discount	Revenue
0	ORD-00001	2023-11-27	CUST-0215	Consumer	Manitoba	Furniture	Lamp	5	205.92	0.10	926.64
1	ORD-00002	2025-08-13	CUST-0386	Consumer	Alberta	Office Supplies	Pens	2	7.37	0.10	13.27
2	ORD-00003	2024-11-30	CUST-0476	Consumer	British Columbia	Office Supplies	Folders	1	54.67	0.05	51.94

Next steps: Generate code with sales_df New interactive sheet

Inspect and Clean the Data

```
print("Dataset shape:", sales_df.shape)
print("\nMissing values:")
print(sales_df.isnull().sum())
print("\nDuplicate rows:", sales_df.duplicated().sum())

sales_df["Order_Date"] = pd.to_datetime(sales_df["Order_Date"])
sales_df = sales_df.drop_duplicates().reset_index(drop=True)

sales_df["Year"] = sales_df["Order_Date"].dt.year
sales_df["Month_Number"] = sales_df["Order_Date"].dt.month
sales_df["Month"] = sales_df["Order_Date"].dt.strftime("%b")
sales_df["Year_Month"] = sales_df["Order_Date"].dt.to_period("M").astype(str)
sales_df["Profit_Margin_Percent"] = np.where(
    sales_df["Revenue"] > 0,
    (sales_df["Profit"] / sales_df["Revenue"]) * 100,
    0
)

print("\nCleaned dataset shape:", sales_df.shape)
sales_df.head()
```

```
Dataset shape: (5000, 13)

Missing values:
Order_ID            0
Order_Date          0
Customer_ID         0
Customer_Segment    0
Region              0
Category            0
Product             0
Quantity            0
Unit_Price          0
Discount            0
Revenue             0
Cost                0
Profit              0
dtype: int64

Duplicate rows: 0

Cleaned dataset shape: (5000, 18)
```

	Order_ID	Order_Date	Customer_ID	Customer_Segment	Region	Category	Product	Quantity	Unit_Price	Discount	Revenue
0	ORD-00001	2023-11-27	CUST-0215	Consumer	Manitoba	Furniture	Lamp	5	205.92	0.10	926.64
1	ORD-00002	2025-08-13	CUST-0386	Consumer	Alberta	Office Supplies	Pens	2	7.37	0.10	13.27
2	ORD-00003	2024-11-30	CUST-0476	Consumer	British Columbia	Office Supplies	Folders	1	54.67	0.05	51.94
3	ORD-00004	2023-06-16	CUST-0841	Consumer	Ontario	Furniture	Cabinet	3	553.55	0.15	1,411.55
4	ORD-00005	2025-08-13	CUST-0367	Consumer	Manitoba	Clothing	T-Shirt	2	108.65	0.00	217.30

Next steps: Generate code with sales_df New interactive sheet

Business KPI Summary

```
total_revenue = sales_df["Revenue"].sum()
total_profit = sales_df["Profit"].sum()
total_orders = sales_df["Order_ID"].nunique()
total_customers = sales_df["Customer_ID"].nunique()
average_order_value = total_revenue / total_orders
profit_margin = (total_profit / total_revenue) * 100

kpi_summary = pd.DataFrame({
    "Metric": [
        "Total Revenue",
        "Total Profit",
        "Total Orders",
        "Total Customers",
        "Average Order Value",
        "Profit Margin (%)"
    ],
    "Value": [
        total_revenue,
        total_profit,
        total_orders,
        total_customers,
        average_order_value,
        profit_margin
    ]
})

kpi_summary
```

	Metric	Value
0	Total Revenue	6,333,819.36
1	Total Profit	1,982,675.05
2	Total Orders	5,000.00
3	Total Customers	990.00
4	Average Order Value	1,266.76
5	Profit Margin (%)	31.30

Next steps: Generate code with kpi_summary New interactive sheet

Monthly Sales Analysis

```
monthly_sales = (
    sales_df.groupby("Year_Month", as_index=False)
    .agg(
        Revenue=("Revenue", "sum"),
        Profit=("Profit", "sum"),
        Orders=("Order_ID", "nunique"),
        Customers=("Customer_ID", "nunique")
    )
)

monthly_sales["Average_Order_Value"] = (
    monthly_sales["Revenue"] / monthly_sales["Orders"]
)

monthly_sales.head()
```

	Year_Month	Revenue	Profit	Orders	Customers	Average_Order_Value
0	2023-01	151,368.99	47,075.76	162	153	934.38
1	2023-02	180,541.37	56,651.76	127	120	1,421.59
2	2023-03	171,390.87	54,906.11	129	124	1,328.61
3	2023-04	150,721.79	48,435.56	131	122	1,150.55
4	2023-05	194,337.78	63,828.01	148	138	1,313.09

Next steps: Generate code with monthly_sales New interactive sheet

Product and Category Performance

```
category_summary = (
    sales_df.groupby("Category", as_index=False)
    .agg(
        Revenue=("Revenue", "sum"),
        Profit=("Profit", "sum"),
        Orders=("Order_ID", "nunique"),
        Quantity_Sold=("Quantity", "sum")
    )
)

category_summary["Profit_Margin_Percent"] = (
    category_summary["Profit"] / category_summary["Revenue"] * 100
)

product_summary = (
    sales_df.groupby(["Category", "Product"], as_index=False)
    .agg(
        Revenue=("Revenue", "sum"),
        Profit=("Profit", "sum"),
        Orders=("Order_ID", "nunique"),
        Quantity_Sold=("Quantity", "sum")
    )
    .sort_values("Revenue", ascending=False)
)

category_summary
```

	Category	Revenue	Profit	Orders	Quantity_Sold	Profit_Margin_Percent
0	Clothing	379,863.23	119,856.05	998	2992	31.55
1	Electronics	4,204,211.83	1,314,713.82	1614	4920	31.27
2	Furniture	1,531,938.03	479,801.29	1169	3495	31.32
3	Office Supplies	217,806.27	68,303.89	1219	3697	31.36

Next steps: Generate code with category_summary New interactive sheet

Regional Performance

```
region_summary = (
    sales_df.groupby("Region", as_index=False)
    .agg(
        Revenue=("Revenue", "sum"),
        Profit=("Profit", "sum"),
        Orders=("Order_ID", "nunique"),
        Customers=("Customer_ID", "nunique")
    )
)

region_summary["Profit_Margin_Percent"] = (
    region_summary["Profit"] / region_summary["Revenue"] * 100
)

region_summary.sort_values("Revenue", ascending=False)
```

	Region	Revenue	Profit	Orders	Customers	Profit_Margin_Percent
2	Manitoba	1,395,231.33	431,506.58	1067	652	30.93
1	British Columbia	1,286,716.13	392,108.35	980	621	30.47
4	Quebec	1,284,291.73	412,859.02	1009	631	32.15
3	Ontario	1,215,429.59	382,061.05	944	605	31.43
0	Alberta	1,152,150.58	364,140.05	1000	640	31.61

Customer Analytics and Segmentation

```
latest_date = sales_df["Order_Date"].max() + pd.Timedelta(days=1)

customer_summary = (
    sales_df.groupby("Customer_ID", as_index=False)
    .agg(
        Last_Purchase_Date=("Order_Date", "max"),
        Total_Orders=("Order_ID", "nunique"),
        Total_Revenue=("Revenue", "sum"),
        Total_Profit=("Profit", "sum"),
        Average_Order_Value=("Revenue", "mean")
    )
)

customer_summary["Recency_Days"] = (
    latest_date - customer_summary["Last_Purchase_Date"]
).dt.days

customer_summary["Customer_Value_Segment"] = pd.qcut(
    customer_summary["Total_Revenue"],
    q=4,
    labels=["Low Value", "Medium Value", "High Value", "VIP"]
)

customer_summary.head()
```

	Customer_ID	Last_Purchase_Date	Total_Orders	Total_Revenue	Total_Profit	Average_Order_Value	Recency_Days	Customer_V
0	CUST-0001	2025-10-29	8	18,776.25	6,609.26	2,347.03	64	
1	CUST-0002	2025-12-04	6	6,929.24	2,460.85	1,154.87	28	
2	CUST-0003	2025-10-22	8	7,438.63	2,250.57	929.83	71	
3	CUST-0004	2025-06-25	5	7,108.12	2,127.35	1,421.62	190	
4	CUST-0005	2025-04-30	6	9,947.30	4,135.32	1,657.88	246	

Next steps: Generate code with customer_summary New interactive sheet

Customer Segment Summary

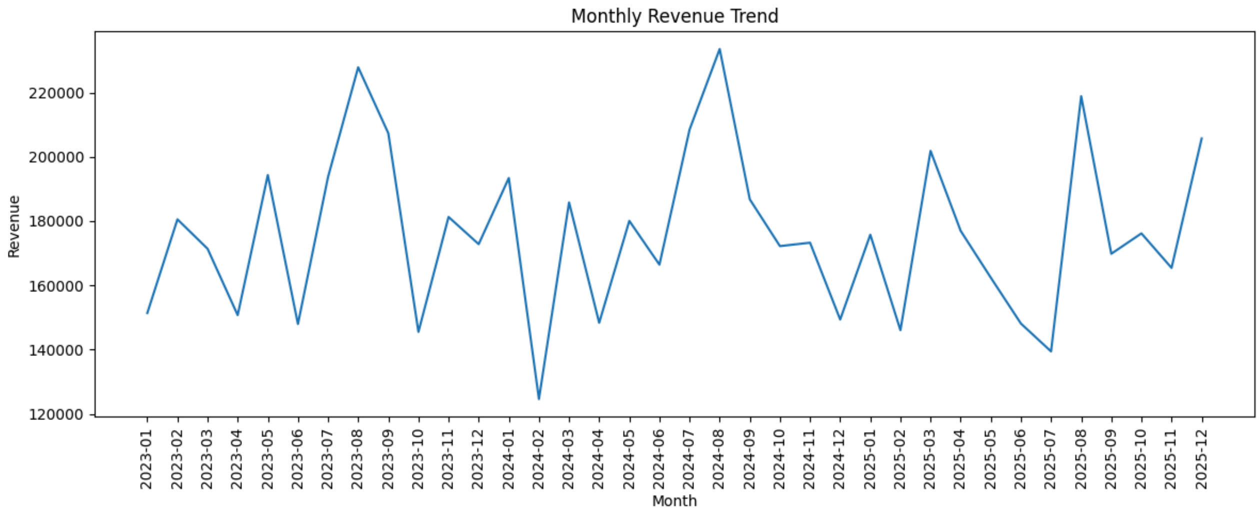
```
customer_segment_summary = (
    customer_summary.groupby(
        "Customer_Value_Segment",
        observed=False,
        as_index=False
    )
    .agg(
        Customers=("Customer_ID", "nunique"),
        Revenue=("Total_Revenue", "sum"),
        Profit=("Total_Profit", "sum"),
        Average_Orders=("Total_Orders", "mean"),
        Average_Recency_Days=("Recency_Days", "mean")
    )
)

customer_segment_summary
```

	Customer_Value_Segment	Customers	Revenue	Profit	Average_Orders	Average_Recency_Days
0	Low Value	248	396,768.31	129,339.18	3.24	307.48
1	Medium Value	247	1,026,510.44	323,135.01	4.67	217.02
2	High Value	247	1,741,702.21	541,493.72	5.50	192.79
3	VIP	248	3,168,838.40	988,707.14	6.79	146.34

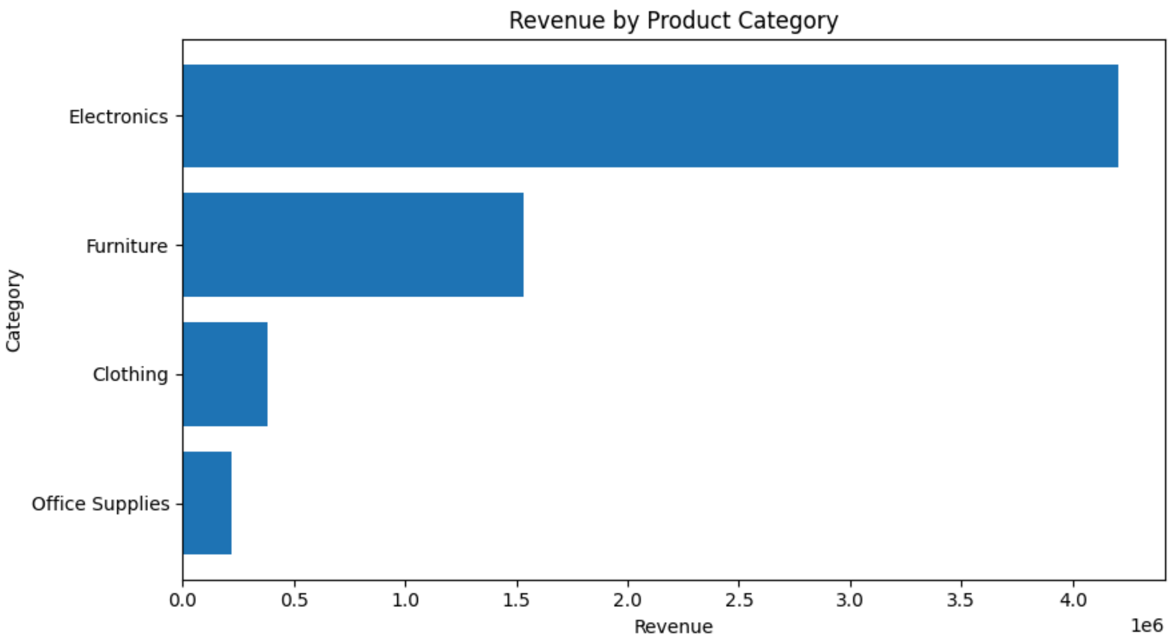
Next steps: Generate code with customer_segment_summary New interactive sheet

Create Portfolio-Quality Charts

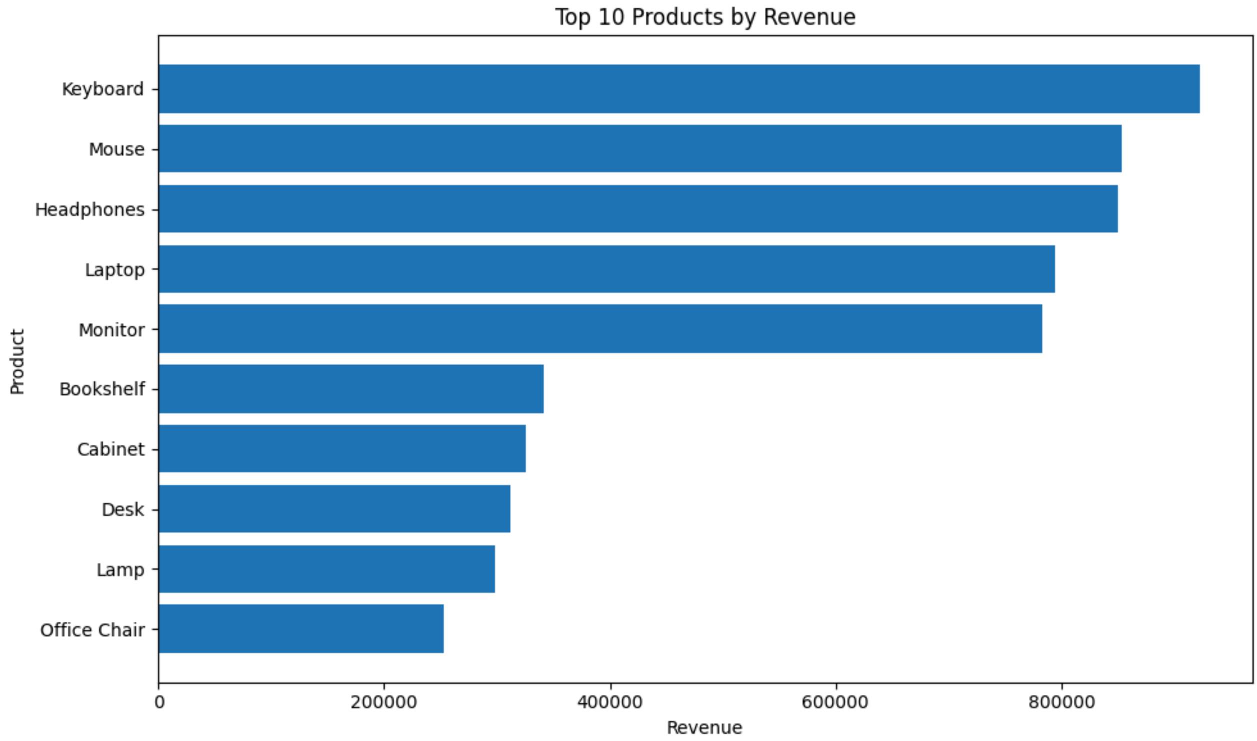
```
plt.figure(figsize=(12, 5))
plt.plot(monthly_sales["Year_Month"], monthly_sales["Revenue"])
plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.xticks(rotation=90)
plt.tight_layout()
plt.savefig("monthly_revenue_trend.png", dpi=300, bbox_inches="tight")
plt.show()
```



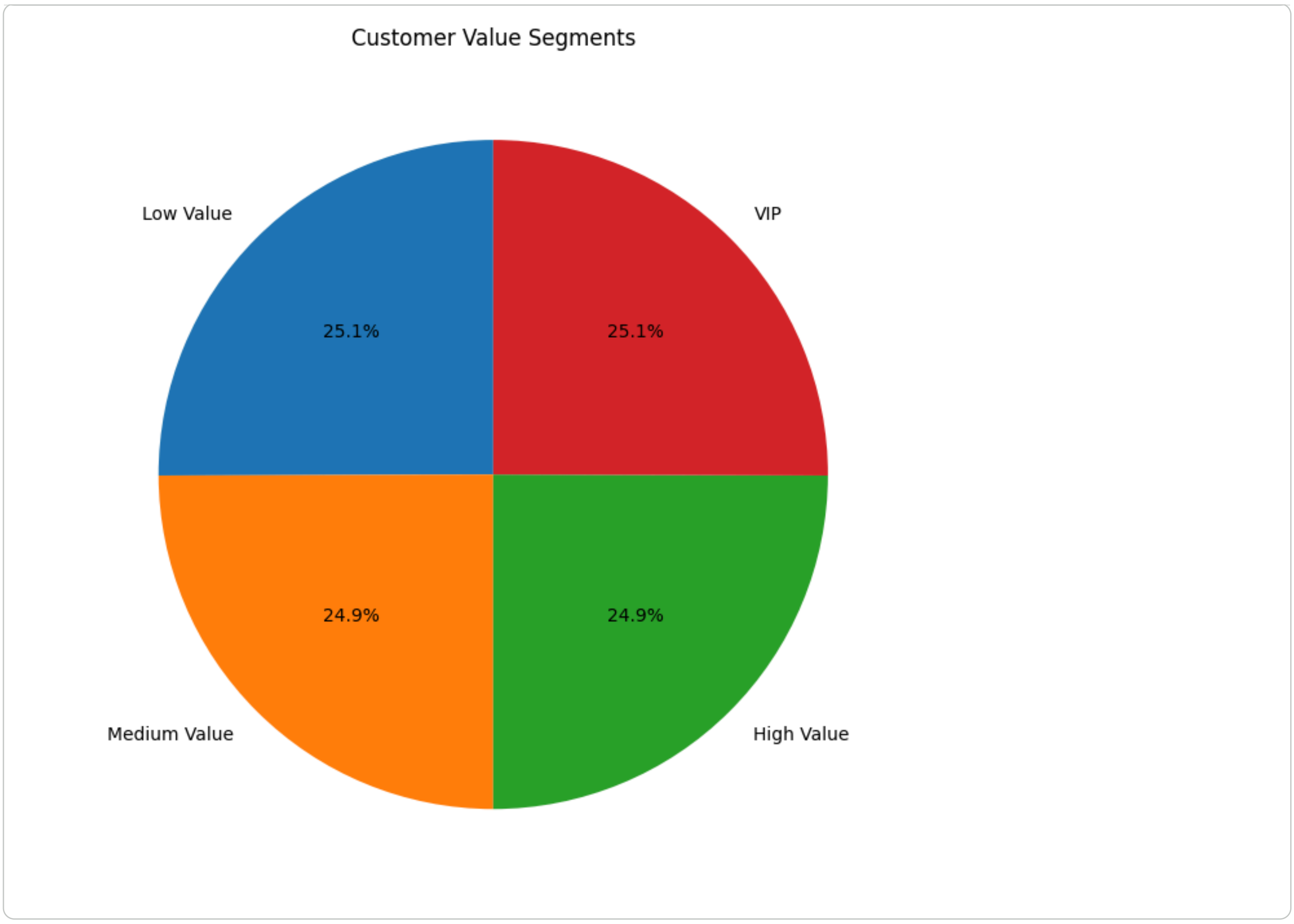

```python
chart_data = category_summary.sort_values("Revenue")

plt.figure(figsize=(9, 5))
plt.barh(chart_data["Category"], chart_data["Revenue"])
plt.title("Revenue by Product Category")
plt.xlabel("Revenue")
plt.ylabel("Category")
plt.tight_layout()
plt.savefig("revenue_by_category.png", dpi=300, bbox_inches="tight")
plt.show()
```




```python
top_products = product_summary.head(10).sort_values("Revenue")

plt.figure(figsize=(10, 6))
plt.barh(top_products["Product"], top_products["Revenue"])
plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product")
plt.tight_layout()
plt.savefig("top_10_products.png", dpi=300, bbox_inches="tight")
plt.show()
```




```
segment_chart = customer_segment_summary.copy()

plt.figure(figsize=(7, 7))
plt.pie(
    segment_chart["Customers"],
    labels=segment_chart["Customer_Value_Segment"],
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Customer Value Segments")
plt.tight_layout()
plt.savefig("customer_segments.png", dpi=300, bbox_inches="tight")
plt.show()
```



Export Files for Power BI

```
sales_df.to_csv("retail_sales_data.csv", index=False)
kpi_summary.to_csv("retail_kpis.csv", index=False)
monthly_sales.to_csv("monthly_sales.csv", index=False)
category_summary.to_csv("category_summary.csv", index=False)
product_summary.to_csv("product_summary.csv", index=False)
region_summary.to_csv("region_summary.csv", index=False)
customer_summary.to_csv("customer_summary.csv", index=False)
customer_segment_summary.to_csv("customer_segment_summary.csv", index=False)

exported_files = [
    "retail_sales_data.csv",
    "retail_kpis.csv",
    "monthly_sales.csv",
    "category_summary.csv",
    "product_summary.csv",
    "region_summary.csv",
    "customer_summary.csv",
    "customer_segment_summary.csv"
]

for file_name in exported_files:
    print(file_name, "- Created" if os.path.exists(file_name) else "- Missing")
```

```
retail_sales_data.csv - Created
retail_kpis.csv - Created
monthly_sales.csv - Created
category_summary.csv - Created
product_summary.csv - Created
region_summary.csv - Created
customer_summary.csv - Created
customer_segment_summary.csv - Created
```

Create SQLite Database

```
connection = sqlite3.connect("retail_sales_analytics.db")

sales_df.to_sql("sales_transactions", connection, if_exists="replace", index=False)
```

```
kpi_summary.to_sql("business_kpis", connection, if_exists="replace", index=False)
monthly_sales.to_sql("monthly_sales", connection, if_exists="replace", index=False)
category_summary.to_sql("category_summary", connection, if_exists="replace", index=False)
product_summary.to_sql("product_summary", connection, if_exists="replace", index=False)
region_summary.to_sql("region_summary", connection, if_exists="replace", index=False)
customer_summary.to_sql("customer_summary", connection, if_exists="replace", index=False)
customer_segment_summary.to_sql(
    "customer_segment_summary",
    connection,
    if_exists="replace",
    index=False
)

connection.close()

```

```
retail_sales_analytics.db - Created
```

Verify SQLite Tables

```
connection = sqlite3.connect("retail_sales_analytics.db")

table_list = pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table';",
    connection
)

connection.close()

table_list
```

	name
0	sales_transactions
1	business_kpis
2	monthly_sales
3	category_summary
4	product_summary
5	region_summary
6	customer_summary
7	customer_segment_summary

Next steps: Generate code with `table_list` New interactive sheet

Recommended Power BI Dashboard

KPI Cards

- Total Revenue
- Total Profit
- Total Orders
- Average Order Value
- Profit Margin

Visuals

- Monthly Revenue Trend — Line chart
- Revenue by Category — Bar chart
- Top 10 Products — Bar chart
- Revenue by Region — Bar chart
- Customer Value Segments — Donut chart
- Sales Transactions — Table

Suggested Power BI Files

Import: